Architecture 1001: x86-64 Assembly SpecialMaths.c with GCC

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

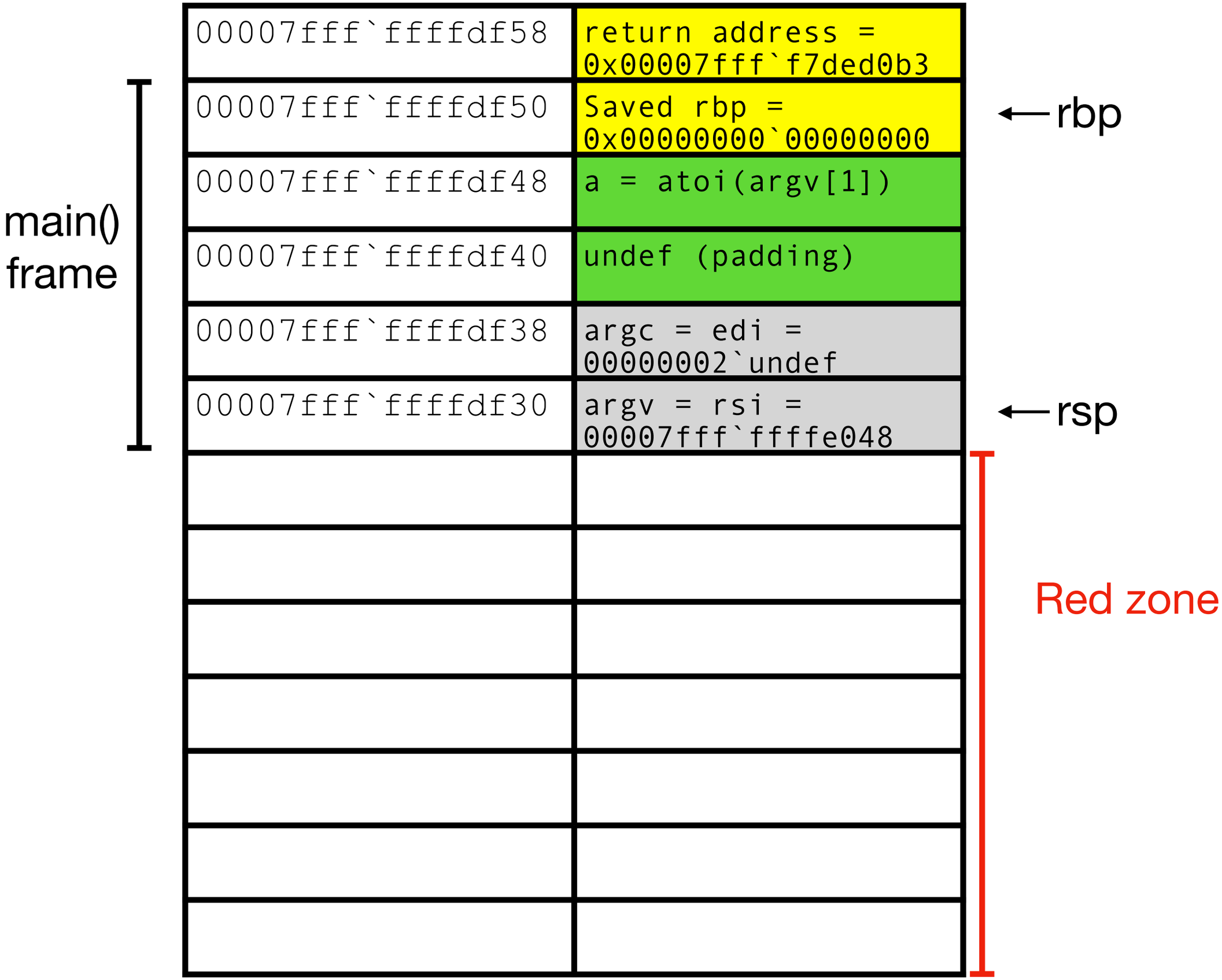
SpecialMaths.c

With command line arguments, 2 function parameters, and special maths known to generate particular asm as a teachable moment :)

```c
#include <stdlib.h>
int main(int argc, char ** argv){
    int a;
    a = atoi(argv[1]);
    return 2*argc + a;
}
```

```
main:
0x0000555555555149 <+0>:   endbr64
0x000055555555514d <+4>:   push   %rbp
0x000055555555514e <+5>:   mov    %rsp,%rbp
0x0000555555555151 <+8>:   sub    $0x20,%rsp
0x0000555555555155 <+12>:  mov    %edi,-0x14(%rbp)
0x0000555555555158 <+15>:  mov    %rsi,-0x20(%rbp)
0x000055555555515c <+19>:  mov    -0x20(%rbp),%rax
0x0000555555555160 <+23>:  add    $0x8,%rax
0x0000555555555164 <+27>:  mov    (%rax),%rax
0x0000555555555167 <+30>:  mov    %rax,%rdi
0x000055555555516a <+33>:  callq  0x555555555050 <atoi@plt>
0x000055555555516f <+38>:  mov    %eax,-0x4(%rbp)
0x0000555555555172 <+41>:  mov    -0x14(%rbp),%eax
0x0000555555555175 <+44>:  lea    (%rax,%rax,1),%edx
0x0000555555555178 <+47>:  mov    -0x4(%rbp),%eax
0x000055555555517b <+50>:  add    %edx,%eax
0x000055555555517d <+52>:  leaveq
0x000055555555517e <+53>:  retq
```

The stack looks like this at line 0x00005555`55555175 in func():



LEAVE - Exit a function

- It's literally just the same thing as the two instructions you'd typically expect to see right before you return from a function that using stack frames:
  - mov rsp, rbp
  - pop rbp


DON'T MAKE IT
UNBALANCED!

Instructions we now know (31)

- NOP
- PUSH/POP
- CALL/RET
- MOV
- ADD/SUB
- IMUL
- MOVZX/MOVSX
- LEA
- JMP/Jcc (family)
- CMP/TEST
- AND/OR/XOR/NOT
- INC/DEC
- SHR/SHL/SAR/SAL
- DIV/IDIV
- REP STOS
- REP MOVS
- **LEAVE**

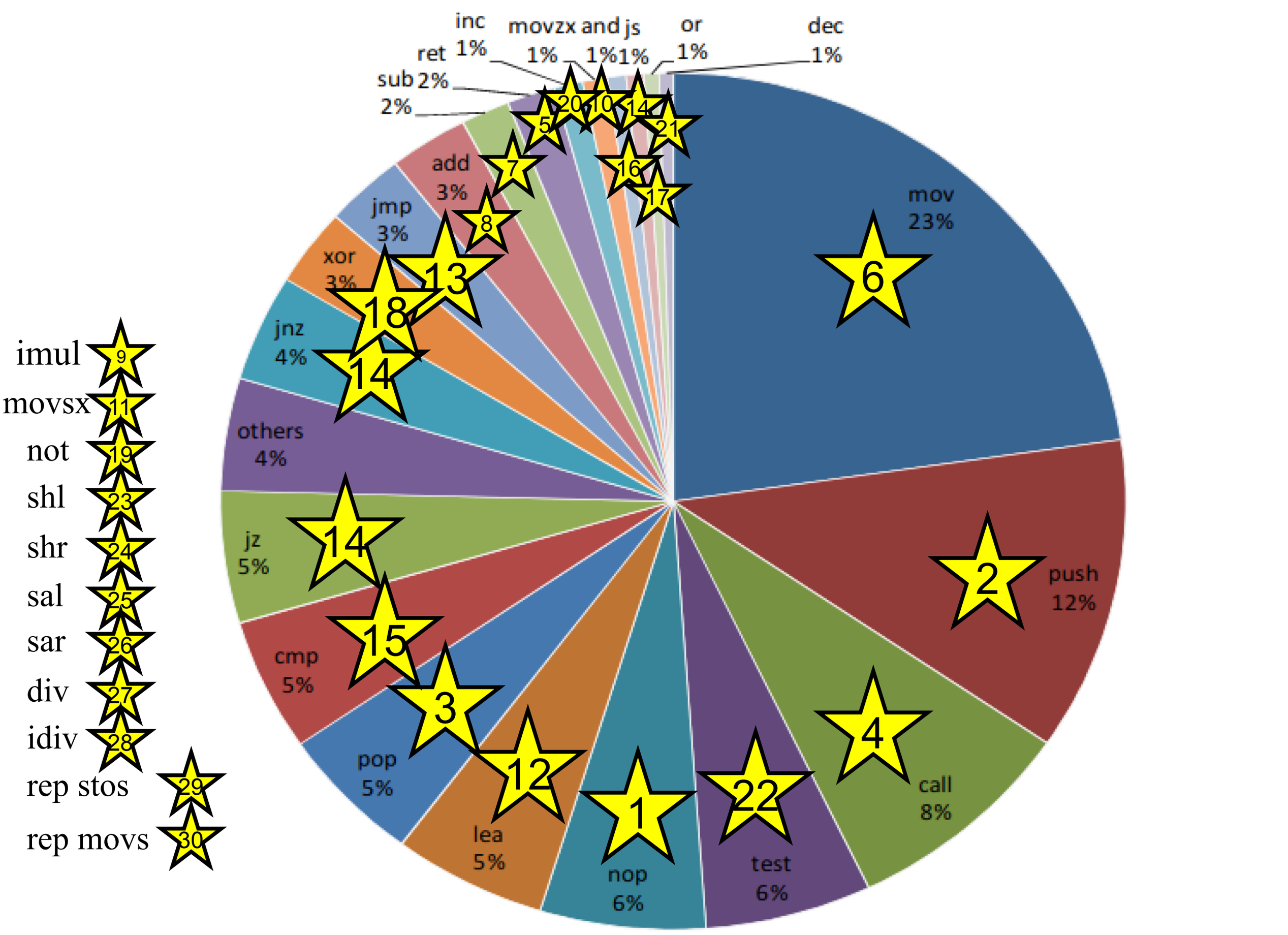


Fig. 2 Web browsers' instruction frequencies

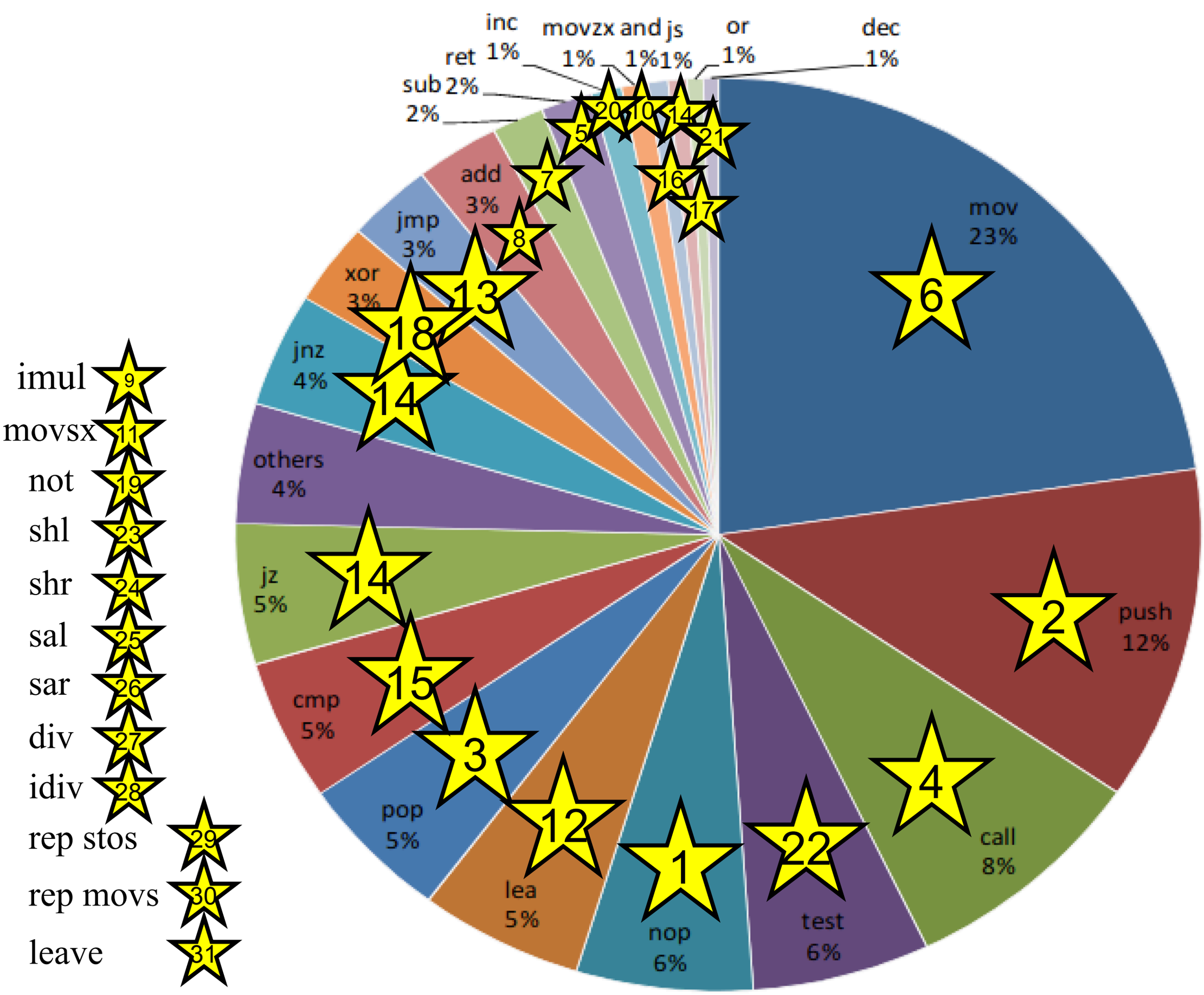


Fig. 2 Web browsers' instruction frequencies